

Lambda/Streams

Methode	Beschreibung	Typische Anwendungsfälle	Wichtige technische Details
<code>allMatch(Predicate<T>)</code>	Prüft, ob alle Elemente die Bedingung erfüllen.	Datenvalidierung, Qualitätsprüfung.	Stoppt sofort, wenn ein negatives Element gefunden wird. Effizient für frühes Scheitern.
<code>anyMatch(Predicate<T>)</code>	Prüft, ob mindestens ein Element die Bedingung erfüllt.	Validierung, Existenzprüfung.	Stoppt sofort, wenn ein passendes Element gefunden wurde. O(n) worst case, O(1) best case.
<code>collect(Collector<T, A, R>)</code>	Aggregation von Stream-Ergebnissen in eine Sammlung oder Statistik.	Gruppierung, Summenbildung, Erstellung von Maps oder Sets.	Collector kann ein mutable reduction durchführen (z. B. <code>toList()</code> , <code>toMap()</code>). Parallelisierbar, wenn der Collector konkurrenzfähig ist.
<code>distinct()</code>	Entfernt doppelte Elemente basierend auf <code>equals()</code> .	Duplikatserkennung, eindeutige Werte extrahieren.	Arbeitet intern mit <code>HashSet</code> . Hoher Speicherbedarf für große Streams. Performance: O(n) bei Hash-basierten Strukturen.
<code>filter(Predicate<T>)</code>	Filtet Elemente basierend auf einer Bedingung.	Datenbereinigung, Entfernen von Null-Werten oder unerwünschten Einträgen.	<code>Predicate<T></code> ist eine funktionale Schnittstelle (<code>T -> boolean</code>). Streams bleiben lazy (Evaluation erfolgt erst bei Terminal-Operationen).
<code>findAny()</code>	Gibt ein beliebiges Element zurück (besonders für parallele Streams optimiert).	Performance-Optimierung bei paralleler Verarbeitung.	Nutzt <code>Optional<T></code> . Kann in parallelen Streams schneller sein als <code>findFirst()</code> .
<code>findFirst()</code>	Gibt das erste Element eines Streams zurück.	Sequentielle Streams mit definierter Reihenfolge.	Nutzt <code>Optional<T></code> . Effizient, da es nach dem ersten Treffer stoppt.
<code>flatMap(Function<T, Stream<R>>)</code>	Entfaltet geschachtelte Strukturen in einen flachen Stream.	Tokenisierung von Texten, Verarbeitung von Listen innerhalb von Listen.	<code>flatMap</code> reduziert eine <code>Stream<Stream></code> -Struktur auf eine Stream. Hohe Rechenintensität bei sehr tief verschachtelten Strukturen.
<code>groupingBy(Function<T, K>)</code> (aus <code>Collectors</code>)	Gruppiert Elemente basierend auf einem Schlüssel.	Kategorisierung, Gruppierung nach Kategorien oder Zeitstempeln.	Erstellt eine <code>Map<K, List></code> . Mit Downstream-Collector auch Aggregationen möglich (z. B. <code>groupingBy(..., counting())</code>).
<code>limit(long n)</code>	Begrenzung auf maximal <code>n</code> Elemente.	Sampling, Zufallsauswahl, Performance-Optimierung.	Effizient bei <code>Stream.of()</code> , <code>List.stream()</code> . Weniger effizient bei <code>generate()</code> oder <code>iterate()</code> . Lazy Evaluation, um nur die ersten <code>n</code> Elemente zu berechnen.
<code>map(Function<T, R>)</code>	Transformiert Elemente von Typ <code>T</code> in Typ <code>R</code> .	Normalisierung von Daten, Umwandlung von Strings in numerische Werte.	<code>Function<T, R></code> ist eine funktionale Schnittstelle (<code>T -> R</code>). Die Pipeline bleibt lazy. Parallele Verarbeitung möglich.
<code>noneMatch(Predicate<T>)</code>	Prüft, ob kein Element die Bedingung erfüllt.	Ausschlusskriterien, Datenvalidierung.	Stoppt sofort, wenn ein passendes Element gefunden wird. O(n) worst case, O(1) best case.
<code>parallel()</code>	Konvertiert einen Stream in einen parallelen Stream.	Performance-Optimierung bei CPU-intensiven Operationen.	Nutzt den <code>ForkJoinPool</code> mit <code>Runtime.getRuntime().availableProcessors()</code> . Achtung: Overhead durch Thread-Switching. Reihenfolge von <code>forEach()</code> kann variieren.
<code>partitioningBy(Predicate<T>)</code> (aus <code>Collectors</code>)	Teilt Elemente in zwei Gruppen (true/false).	Binäre Klassifikation, Trennen von gültigen und ungültigen Daten.	Erstellt eine <code>Map<Boolean, List></code> . Kann mit Downstream-Collector weiterverarbeitet werden. Effizienter als <code>groupingBy()</code> für binäre Werte.
<code>peek(Consumer<T>)</code>	Führt eine Nebenwirkung aus, ohne den Stream zu modifizieren.	Debugging, Logging, Zwischenschritte in Datenpipelines.	Nicht empfohlen für produktive Zwecke außer für Debugging. Kein garantierter Aufruf in parallelen Streams.
<code>reduce(BinaryOperator<T>)</code>	Aggregiert Werte zu einem einzigen Ergebnis.	Berechnung von Summen, Mittelwerten, Maximum/Minimum.	Funktioniert nach Associativity & Identity Law. Falls <code>reduce(identity, accumulator)</code> genutzt wird, kann der Stream leer sein, ohne Exception. Bei <code>reduce(accumulator)</code> liefert ein leerer Stream <code>Optional<T></code> .
<code>sequential()</code>	Erzwingt sequenzielle Verarbeitung eines Streams.	Rückkehr zu deterministischer Reihenfolge nach <code>parallel()</code> .	Kann teuer sein, wenn bereits ein parallelisierter Stream genutzt wurde.

Methode	Beschreibung	Typische Anwendungsfälle	Wichtige technische Details
<code>skip(long n)</code>	Überspringt die ersten <code>n</code> Elemente.	Implementierung von Paging, Entfernen irrelevanter Daten.	Lazy Evaluation. Effizient bei Arrays und Listen, aber potenziell teuer bei <code>generate()</code> oder <code>iterate()</code> .
<code>sorted(Comparator<T>)</code>	Sortiert Elemente gemäß <code>Comparator<T></code> .	Anordnung nach Zeitstempel, numerische Reihenfolgen.	Falls kein <code>Comparator</code> angegeben, wird natürliche Ordnung (<code>Comparable<T></code>) genutzt. In parallelen Streams nutzt <code>sorted()</code> eine stabile Mergesort-Variante.